Transformers in NLP

Concept-first guide with real-world examples

Based on the group paper Transformers and Their Application in Natural Language Processing (NLP) Tasks (Vilnius Gediminas Technical University, Engineering of Artificial Intelligence).

> **How to use this guide**
> Read one concept at a time. For each one: (1) read the real-world picture so the idea feels familiar, (2) read how the idea works, (3) work through the example, (4) answer the self-check before reading on. Answers to the self-checks are at the end. Numbers in the examples come from the paper's results, or are simple made-up values chosen to be easy to follow.

Part	What you will understand
1	The problem: teaching a computer to read a review and judge its sentiment
2	Why language is hard for computers
3	The older methods: word counting, classic classifiers, word vectors, recurrent networks
4	The Transformer, piece by piece: attention, positions, layers
5	How Transformers learn: pre-training, fine-tuning, loss, optimiser, distillation
6	How the experiment is measured: metrics, statistics, data efficiency, cost
7	One sentence followed through the whole system
8	Final self-test and answers

Part 1. The problem

Imagine you run an online shop with 50,000 customer reviews a month. You want a program that reads each review and decides whether the customer is happy or unhappy, so that unhappy customers can be contacted quickly. Reading all 50,000 yourself is not realistic. This is **sentiment classification**: putting a piece of text into one of a fixed set of categories based on how it feels.

> **Real-world picture**
> A film critic reads a review and writes “thumbs up” or “thumbs down”. The program must do the same thing, but it cannot understand a sentence the way a person does. It can only work with numbers. The whole field of NLP (natural language processing) is about turning text into numbers in a way that keeps its meaning.

The task in the paper. Each sentence is a movie review sentence from the Stanford Sentiment Treebank (SST-2). The program outputs one of two labels: 0 for negative, 1 for positive. A test set of 872 sentences is held back and used only for the final score, so the result is fair.

Example sentences

Sentence	Label	Why it is interesting
“A gorgeous, witty, and poignant film.”	positive	Clear positive words, easy case
“The plot is not good.”	negative	The word good is positive, but “not” flips it
“The plot is weak, but the acting is great.”	positive overall	Two opinions joined by a contrast word
“Not bad at all.”	positive	Negative-looking words, positive meaning

The five research questions, in plain words

- **RQ1 (accuracy).** Are Transformers more accurate than older methods on the same data?
- **RQ2 (cost).** What do they cost in time, speed and size on an ordinary laptop?
- **RQ3 (data).** How does the advantage change when we have few labelled examples?
- **RQ4 (ready-made).** How do models that are already trained for sentiment compare with ones we train?
- **RQ5 (hard cases).** Are Transformers better on sentences with negation and contrast?

Part 2. Why language is hard

Concept 1: Word order changes meaning

Real-world picture

“The dog bites the man” and “The man bites the dog” contain exactly the same words. A news headline depends on who did what to whom. A shop assistant reading a list of words “good, not, service, slow” could not tell whether the customer was happy.

The idea. A program that only counts words treats both sentences as identical. A program that reads word order can tell them apart. This is why the Transformer adds a position signal to each word (Concept 8).

Worked example

Word counts for both sentences: dog 1, bites 1, man 1, the 2. The counts are identical, so a counting method cannot separate them.

Check yourself

Why can a program that only counts words not separate “dog bites man” from “man bites dog”?

Concept 2: Negation and contrast

Real-world picture

If a friend says “It was not bad”, you understand “it was okay, even good”. If they say “It was bad, but the staff were kind”, you understand that the staff part is positive and the overall feeling is mixed. Small words like “not” and “but” carry a lot of meaning.

The idea. The paper tests this directly. Its negation subset contains words such as not, no, never, n't, nothing, nor and neither. Its contrast subset contains but, although, though, yet and however. The Transformer's real advantage shows up on these sentences (Part 6).

Worked example

“The film is good.” → positive. “The film is not good.” → negative. Same content word, opposite label. Only the small word “not” and its position tell the program which is which.

Check yourself

In “The food was cold, but the staff were lovely”, which part is positive, and which part is negative?

Part 3. The older methods

The Transformer makes more sense once you see what it replaced. Each older method solved part of the problem and left a gap that the next method filled.

Concept 3: Counting words: bag of words and TF-IDF

> **Real-world picture**
> A spam filter in an email inbox. It counts how often words like “free”, “winner” and “click” appear. A lot of such words suggests spam. A search engine does something similar: pages with your search words rank higher, and rare words count more than common ones.

The idea. Each sentence becomes a long vector with one entry per word. TF-IDF stands for term frequency times inverse document frequency. A word gets a high weight if it appears often in this sentence but in few sentences overall. Words found everywhere, like “movie”, get weight zero because they say nothing about the difference between sentences.

> **Worked example**
> Three mini-reviews: s1 = “good movie”, s2 = “bad movie”, s3 = “not good movie”. The word “movie” appears in all three, so its weight is 0. The word “not” appears only in s3, so it gets a high weight (about 1.10). The word “good” appears in two sentences, so it gets a medium weight (about 0.41).

> **Check yourself**
> Why does the word “movie” get weight 0 in this example? What would happen to a classifier that relied only on these weights?

Concept 4: Classic classifiers: naive Bayes, logistic regression, SVM

> **Real-world picture**
> A bank decides whether to approve a loan. It gives points for each fact (income, past payments, debt), adds them up, and checks whether the total passes a line. That is the same idea as a linear classifier. A spam filter using naive Bayes asks: “how likely is this word in spam compared with normal email?” and multiplies the evidence from each word.

The idea. Naive Bayes multiplies word probabilities and assumes words do not affect each other. Logistic regression adds weighted word scores and passes the total through an S-shaped curve, so the result is a probability between 0 and 1. A linear SVM finds the straight dividing line with the widest safety gap between the two groups. All three are fast and cheap, and all three can only draw straight boundaries over word counts.

> **Worked example**
> Logistic regression with word weights good = +2 and not = 0 scores “not good” as +2, which is positive, and gets it wrong. If we add a feature for the pair “not good” with weight −3, the score becomes 2 + (−3) = −1, and the curve gives 0.27, which is negative and correct. This is why the paper also uses word pairs (bigrams) as features.

> **Check yourself**
> Why does the logistic regression score for “not good” need the pair feature, not just the single words?

Concept 5: Word vectors: giving words a position on a map

> **Real-world picture**
> Think of a map of cities. Vilnius and Kaunas are close together, and Paris is far away. Distance on the map tells you something real about the places. Word vectors do the same for words: words used in similar ways end up close together in a space of many numbers.

The idea. Each word is represented by a vector of numbers, learned from large amounts of text (word2vec, 2013). Similar words get similar vectors. The limitation: each word has only one vector. The word “bank” gets one point on the map, even though it means a river edge and a money institution.

> **Worked example**
> Toy 2D vectors (made up for illustration): coffee = [0.9, 0.1], tea = [0.8, 0.2], car = [0.1, 0.9]. Coffee and tea point in almost the same direction (cosine similarity about 0.99), while coffee and car are far apart (about 0.22). So the model knows coffee is closer to tea than to car.

> **Check yourself**
> If “bank” has only one vector, how can a model tell “river bank” from “bank account”?

Concept 6: Recurrent networks (RNN, LSTM, BiLSTM): reading one word at a time

> **Real-world picture**
> The telephone game. A message passes from person to person, and each person repeats what they heard. Details far from the start get lost. Or a student reading a long book and taking notes: they keep a short summary in their head and update it word by word.

The idea. An RNN reads a sentence from left to right, carrying a memory vector forward. An LSTM adds gates that decide what to keep and what to forget, so the memory lasts longer. A bidirectional LSTM (BiLSTM) reads the sentence twice, left to right and right to left, and combines both. The drawback: each step must wait for the previous one, like cars on a single-lane road, so training is slow and long-distance links are weak.

> **Worked example**
> In “The plot that the critics praised for years was dull”, the word “dull” must connect back to “plot” across many words. A step-by-step memory may have faded by then. A Transformer can link them directly (Part 4).

> **Check yourself**
> Why can a recurrent network not process all the words of a sentence at the same time?

Part 4. Inside the Transformer

The Transformer (Vaswani et al., 2017) removed the step-by-step memory. Every word looks directly at every other word, and all words are processed together. The next six concepts build it up one piece at a time.

Concept 7: Tokenisation: splitting words into reusable pieces

> **Real-world picture**
> Lego bricks. A small set of bricks builds a huge number of models. Lithuanian makes this clear: “namas” (house), “namai” (houses), “namuose” (in the houses) share the root “nam-”. A model that stores the pieces can handle new forms it has never seen.

The idea. Common words stay whole. Rare words are split into known pieces, so the vocabulary is about 30,000 pieces for BERT and no word is ever completely unknown. A special [CLS] token is placed at the start of each input and [SEP] at the end.

> **Worked example**
> The paper's example: “unwatchable” becomes “un” + “##watch” + “##able”. The two “#” marks show that these pieces continue a word.

> **Check yourself**
> Why is it useful to split “unwatchable” into three pieces instead of storing every possible word?

Concept 8: Embeddings and positional encoding: what the word is, and where it sits

> **Real-world picture**
> Seat numbers in a cinema. Two people with the same name can sit in different seats. Without seat numbers, the usher could not tell which person is in row 1. A sentence works the same way: “dog bites man” and “man bites dog” use the same words in different seats.

The idea. Each token id is looked up in an embedding table and becomes a vector of size d (128 for BERT-tiny, 512 for BERT-small, 768 for DistilBERT). Self-attention alone ignores order, so a position vector is added to each token vector. The original Transformer used fixed wave patterns; BERT learns one position vector for each place up to 512.

> **Worked example**
> Input vector for the word at position t = (word vector) + (position vector). The same word “bites” at position 2 and at position 3 gets different input vectors, so the model can tell them apart.

> **Check yourself**
> Why would a Transformer without position information treat “man bites dog” the same as “dog bites man”?

Concept 9: Self-attention: each word asks which other words matter

Real-world picture

A reader stuck on a word in a long document looks back at the earlier sentences that explain it. The word “it” looks back to find what “it” refers to. Or a search box: your question (query) is compared with the labels on every book (keys), and you read the books whose labels match most closely (values).

The idea. Each token makes three vectors: a **query** (what am I looking for?), a **key** (what do I offer?) and a **value** (what information do I pass on?). Queries are compared with every key using a dot product. A softmax turns the scores into weights that are positive and add up to 1. Each token's new vector is the weighted average of all the value vectors. The result: every word gets information from every other word, whatever the distance.

Worked example: two tokens, “not” and “good”

Use very small vectors and let Q = K = V = the input, to keep the arithmetic readable:

• “not” = [1, 0] and “good” = [0, 1].

• **Scores** $Q \cdot K^T$: the diagonal is 1, the rest is 0. Divide by $\sqrt{2} \approx 1.414$, so the diagonal becomes 0.707.

• **Softmax** for the “not” row: $e^{0.707} = 2.03$ and $e^0 = 1$. Weights = 2.03/3.03 = **0.67** and 1/3.03 = **0.33**.

• **New vector for “not”** = 0.67 × [1, 0] + 0.33 × [0, 1] = **[0.67, 0.33]**.

• **New vector for “good”** = 0.33 × [1, 0] + 0.67 × [0, 1] = **[0.33, 0.67]**.

Meaning: after attention, “good” now carries a piece of “not”. In a trained model the matrices are learned, so the model can learn that this mixing is useful for sentiment.

Check yourself

In your own words, what does each of query, key and value represent in the library example?

Why divide by √d? Dot products grow as vectors get longer. Very large scores make the softmax put almost all its weight on one word, and the learning signal becomes nearly zero. Dividing by √d keeps the scores in a useful range. Why is it called self-attention? Because the sentence attends to itself.

Concept 10: Multi-head attention: several readers with different checklists

Real-world picture

A panel reviewing a grant application. One reviewer checks the budget, another the scientific idea, another the writing. Each reads the same text, but looks for different things. Their notes are combined into one report.

The idea. Instead of one attention, the Transformer runs several (called heads) in parallel. Each head has its own query, key and value matrices, working in a smaller space (d / h). Their outputs are joined together. The paper gives the number of heads: 2 for BERT-tiny, 8 for BERT-small, 12 for DistilBERT. Research has found heads that specialise, for example one linking adjectives to nouns and another linking negation words to what they negate.

Worked example

With d = 768 and h = 12 heads (DistilBERT), each head works with vectors of size 768 / 12 = 64. Twelve heads of size 64 together use the same total size as one head of size 768.

Check yourself

If a model has 12 heads of size 64, how big is each head's working space, and why is the total not 12 times bigger?

Concept 11: Feed-forward layers, residual connections and layer normalisation

Real-world picture

Editing a document with track changes. The original text stays visible underneath, and your edits are added on top, so nothing is lost. After each edit, a copy editor tidies up the layout so the document stays readable.

The idea. After attention, each token passes through a small two-layer network separately (the feed-forward layer). The residual connection adds the input back to the output, so the original information is never fully lost and gradients can travel through many layers. Layer normalisation rescales each vector so its numbers stay in a stable range.

Worked example

Input x = [2, 4]. Suppose the attention output adds [0.5, −1]. Residual result = [2.5, 3]. The layer norm then rescales this vector so its mean is 0 and its spread is 1 (before learned scale and shift).

Check yourself

Why is it helpful that the input is added back to the output, instead of replacing it?

Concept 12: Stacking layers: refining the meaning step by step

Real-world picture

A translation passed through several editors. The first editor fixes spelling, the second fixes grammar, the third improves style. Each one works on the previous version.

The idea. The encoder is a stack of identical layers. Each layer contains attention and a feed-forward block. Early layers tend to capture simple patterns (word pairs and local structure), and later layers capture more complex relations. The number of layers in the paper: 2 for BERT-tiny, 4 for BERT-small, 6 for DistilBERT, 12 for BERT-base.

Size comparison (from the paper)

Model	Layers	Hidden size	Heads	Parameters
BERT-tiny	2	128	2	4.4 million
BERT-small	4	512	8	28.8 million
DistilBERT	6	768	12	67.0 million
BERT-base (teacher)	12	768	12	about 110 million

Check yourself

Why would one layer alone be less capable than six stacked layers?

Concept 13: The [CLS] token: one slot that holds the sentence summary

Real-world picture

A class folder with a cover sheet. Each student's work goes inside, and the teacher reads only the cover sheet to give the grade. The [CLS] token is that cover sheet.

The idea. A special [CLS] token is added at the start of every input. After all the layers, its final vector is used as the summary of the whole sentence. Other tokens help it by attending to them during every layer.

Worked example: the full path of one review

“The plot was not good.” becomes: [CLS] the plot was not good [SEP]. Each token gets an embedding plus a position vector. Layers of attention and feed-forward mix the information. The vector at [CLS] now summarises the sentence, and a small output layer turns it into two numbers (for negative and positive).

Check yourself

Why does the model use one special token for the summary, instead of averaging all the words?

Part 5. How Transformers learn

A Transformer starts with random numbers. Learning means adjusting millions of numbers so that predictions match the right answers. This part explains the two-stage approach the paper relies on.

Concept 14: Pre-training: learning language from unlabelled text

> **Real-world picture**
> A child learns English by listening and reading, long before anyone teaches grammar rules. Or a fill-in-the-blank exercise in a language course: “The train arrived ___ time.” You can answer without knowing the topic, because you know how English works.

The idea. Pre-training hides about 15% of the words in a large text collection and asks the model to predict the hidden words from the context on both sides. This is called masked language modelling (MLM). It needs no labels, so it can use billions of sentences. Pre-training is extremely expensive (days on many GPUs), so the paper uses public pre-trained weights and does not repeat it.

> **Worked example**
> Input: “The movie was really [MASK].” The model's job is to give high probability to words like “good” or “bad”, using the context “movie” and “really”. Doing this across billions of sentences teaches grammar, meaning and common facts.

> **Check yourself**
> Why does pre-training not need any sentiment labels to be useful for sentiment classification?

Concept 15: Fine-tuning: a short, focused training on your own task

> **Real-world picture**
> A chef who has trained for ten years is hired by a new restaurant. They do not need to learn cooking again. They need only a few weeks to learn this restaurant's menu and style.

The idea. The pre-trained model gets one small new layer on top, with two outputs (negative and positive). The whole network is then trained on the labelled sentences, usually for only 2 or 3 passes over the data. The paper fine-tunes on 8,000 labelled sentences, which is about 12% of the training data.

> **Worked example**
> BERT-small fine-tuned on 8,000 sentences reached 83.7% on the test set. A TF-IDF model trained from scratch on all 65,349 sentences reached 81.9%. The pre-trained model needed far fewer labels to reach a similar level.

> **Check yourself**
> What is the difference between pre-training and fine-tuning, in one sentence each?

Concept 16: Cross-entropy loss: scoring how confident and how right the model was

> **Real-world picture**
> A weather forecaster who says “90% chance of rain” and it rains gets a small penalty. Who says “20% chance” and it rains gets a large penalty. Forecasters are scored on the probability they gave to what actually happened.

The idea. The loss is −log(probability of the correct answer). Probabilities close to 1 give a loss close to 0. Probabilities close to 0 give a very large loss. Training changes the model's numbers to make the average loss over all training sentences smaller.

> **Worked example**
> True label = positive. Model gives P(positive) = 0.9 → loss = −ln(0.9) = 0.105 (small). Model gives P(positive) = 0.2 → loss = −ln(0.2) = 1.61 (large).

> **Check yourself**
> A model says P(positive) = 0.5 for a positive review. Is its loss higher or lower than when it says 0.9? Why?

Concept 17: Optimiser, learning rate, warm-up and clipping: how big each correction step is

> **Real-world picture**
> Learning to drive. At first you go slowly and carefully, because a sudden swerve could be dangerous. Once you are used to the car, you can accelerate more smoothly. A speed limiter also stops you from accelerating too sharply in any one moment.

The idea. The learning rate is the step size for each correction. AdamW is the optimiser: it adapts the step size for each number, using running averages of past gradients, and includes weight decay that gently pulls numbers toward zero. Warm-up starts with a tiny learning rate and increases it over the first 10% of training, protecting the pre-trained knowledge from being damaged early. Gradient clipping (maximum norm 1.0) limits how large any single correction can be.

> **Worked example**
> The paper's learning rates: 2×10^{-4} for BERT-tiny, 1×10^{-4} for BERT-small, 5×10^{-5} for DistilBERT. Smaller models get larger steps, and the large pre-trained DistilBERT gets the gentlest steps.

> **Check yourself**
> Why does the pre-trained DistilBERT get a smaller learning rate than BERT-tiny?

Concept 18: Distillation: a small student copies a large teacher

> **Real-world picture**
> A master chef teaches an apprentice by letting them taste every dish and copy the taste, not just the recipe. The apprentice can cook most of the menu in a fraction of the time.

The idea. DistilBERT is trained to imitate the output of BERT-base, its teacher, rather than only the correct labels. The result is a model that is 40% smaller and 60% faster, and keeps about 97% of BERT's language understanding performance.

Worked example

Teacher has 12 layers. Student has 6 layers. The student is about 40% smaller and about 60% faster, and keeps about 97% of the teacher's score on language-understanding benchmarks.

Check yourself

Why would training a small model to copy a big model work better than training the small model from scratch?

Part 6. How the experiment is measured

Good research measures in a way that can be checked. This part covers the tools the paper uses to show which results are real.

Concept 19: Train, dev and test: practice exam, mock exam, final exam

Real-world picture

A student studies with practice papers (training), checks progress with a mock exam (dev set), and then sits the real exam once (test set). If the student kept taking the real exam to adjust their revision, the final score would look better than it really is.

The idea. The paper holds out 2,000 training sentences as a dev set for choosing the best training epoch, and uses the official 872-sentence validation set as the final test set. Each model is tested once. Settings are fixed in advance and are not tuned on the test set.

Check yourself

Why must the test set never be used to pick the best epoch?

Concept 20: Confusion matrix, accuracy, precision, recall and F1

Real-world picture

An airport security scanner or a medical test. Some people flagged are innocent (false alarm). Some dangerous cases are missed (miss). Accuracy alone hides this difference, so we count the two kinds of mistake separately.

The idea. Precision asks: of the sentences predicted positive, how many really were? Recall asks: of the truly positive sentences, how many did we find? F1 combines both into one number. Macro F1 averages the positive and negative classes equally.

Worked example: 100 test sentences

	Predicted positive	Predicted negative
Actually positive	TP = 45	FN = 5
Actually negative	FP = 10	TN = 40

• Accuracy = (45 + 40) / 100 = **85%**.
• Precision (positive) = 45 / 55 = **81.8%**: when the model says positive, it is right 82% of the time.
• Recall (positive) = 45 / 50 = **90%**: it finds 90% of the real positives.
• F1 (positive) = 2 × 0.818 × 0.90 / (0.818 + 0.90) = **85.7%**.

Check yourself

A spam filter has high recall but low precision. What does that mean for the user?

Concept 21: Bootstrap confidence intervals: how stable is the score?

Real-world picture

Political polling. A poll of 1,000 people gives 52% support, but pollsters report a margin of error. To estimate it without many new polls, you can imagine resampling the same respondents with replacement, many times, and see how much the result moves.

The idea. Resample the 872 test sentences with replacement 2,000 times (some sentences appear twice, some not at all). Compute accuracy on each resample. The middle 95% of those 2,000 accuracies is the confidence interval. It shows how much the score could change if the test set were a bit different.

Worked example

DistilBERT fine-tuned on 8,000 sentences: accuracy 88.8%, interval [86.6%, 90.7%]. Its true accuracy on similar sentences is likely between 86.6% and 90.7%, so a score of 81.9% is clearly below it.

Check yourself

If two models' intervals overlap a lot, what can we say about their difference?

Concept 22: McNemar's test: comparing two models on the same sentences

Real-world picture

Two doctors look at the same 872 patient files and make a diagnosis. Most files are handled the same way by both. The useful evidence is in the files where exactly one doctor is right. If one doctor is right on many more files that the other gets wrong, that is evidence of a real difference. If the disagreements are balanced, the difference is probably random.

The idea. Count the sentences where only model A is right (b) and where only model B is right (c). If the two models were equally good, b and c would be about equal. The test checks whether the imbalance is larger than chance would produce.

Worked example: the paper's DistilBERT vs DistilBERT-SST2 comparison

Only the best model is right on b = 48 sentences; only the fine-tuned model is right on c = 28 sentences.

Continuity-corrected chi-square = $(|48 - 28| - 1)^2 / (48 + 28) = 19^2 / 76 = 4.75$, which gives $p \approx 0.029$. This is below 0.05, so the difference is statistically significant.

Check yourself

Why is it better to compare the models on the same sentences than to compare their overall scores?

Concept 23: Learning curves: how much labelled data is enough?

Real-world picture

Learning a language or practising for a driving test. The first few hours matter the most. Later, extra practice helps less and less. The learning curve shows this: performance against the amount of practice.

The idea. The paper trains both TF-IDF and BERT-small on 500, 1,000, 2,000, 4,000 and 8,000 sentences, using the same examples each time. The gap between them is largest at the smallest size, which is the point of data efficiency: how much accuracy you get per labelled example.

Worked example

Labelled sentences	BERT-small %	TF-IDF + LR %	Gap (points)
500	75.9	61.8	+14.1
1,000	77.6	68.1	+9.5
2,000	80.7	71.0	+9.7
4,000	82.7	75.8	+6.9
8,000	83.7	77.5	+6.2

Check yourself

If you can label only 500 reviews, which model would you choose, and why?

Concept 24: Latency, model size and cost: speed and memory trade-offs

Real-world picture

A postal sorting centre. A machine that reads addresses in a fraction of a second can handle millions of letters, but it needs a large and expensive building. A small hand-sorting team is cheap to set up but slow for large volumes. The right choice depends on the volume.

The idea. Latency is the time to classify one sentence. Model size is the number of parameters, which decides memory and storage. Training time is the cost of building the model. The paper reports all three alongside accuracy, because accuracy alone does not tell you whether a model fits your situation.

Worked example

TF-IDF + linear SVM: 0.06 ms per sentence, about 74 thousand weights, 2 seconds of training. DistilBERT: 39 ms per sentence (about 650 times slower), about 67 million parameters, about 2.6 hours of CPU training.

Check yourself

A company must classify 10 million reviews per second. Which model family is more suitable, and why?

Concept 25: Zero-shot classification: classifying without examples

> **Real-world picture**
> A new employee is given a short description of each category (“complaints about delivery”, “praise for staff”, “questions about price”) and asked to sort a pile of emails on the first day, with no training examples.

The idea. A model trained on natural language inference (NLI) decides whether one sentence (the premise) supports another (the hypothesis). The review is the premise. For each label, a hypothesis is written, such as “The sentiment of this movie review is positive.” The label whose hypothesis gets the highest support wins. No sentiment-labelled training data is needed.

> **Worked example**
> Premise: “The plot was not good, but the acting was brilliant.” Hypothesis for positive: “The sentiment is positive.” Hypothesis for negative: “The sentiment is negative.” The model scores each, softmax turns the scores into probabilities, and the top one is chosen. In the paper this scored 81.8%, but it takes 107 ms per sentence.

> **Check yourself**
> Why is the zero-shot model slower than the fine-tuned models, even though it was never trained on this task?

Concept 26: Overfitting and generalising: studying the wrong practice set

> **Real-world picture**
> A student who memorises the answers to a practice paper, then fails a real exam with different questions. The practice score was high, but it did not show real understanding.

The idea. The paper's BiLSTM reached 91.3% on the dev set, which contains short phrases like the training data, but only 82.8% on the test set of complete sentences. The model learned the style of short phrases better than the general skill. Training sentences are short, so a model can look good on them without generalising.

> **Worked example**
> BiLSTM dev accuracy 91.3%, test accuracy 82.8%. The gap of about 8.5 points is a warning that the model fits its training style too closely.

> **Check yourself**
> Why is a high dev score not enough to show that a model will work on new sentences?

Part 7. One sentence, the whole system

Follow “The plot was not good, but the acting was brilliant” through the complete Transformer system.

Step	What happens	Concept
1. Tokenise	Split into pieces and add [CLS] at the start and [SEP] at the end	7
2. Embed	Look up a vector for each piece, then add a position vector	8
3. Attend	Each piece looks at all others. “not” can influence “good”; “but” can mark a change of direction	9, 10
4. Refine	Feed-forward layers, residual connections and normalisation, repeated across 6 layers for DistilBERT	11, 12
5. Summarise	Read the final [CLS] vector, which now summarises the whole sentence	13
6. Decide	A small output layer gives two scores, and softmax turns them into probabilities	16
7. Output	Positive probability is 1.000 for this sentence in the paper's example; the decision is positive	15

Why it got this right while the SVM did not

A bag-of-words model sees “not”, “good”, “brilliant” as separate features and cannot tell which clause each word belongs to. The Transformer connects “but” to the clause after it, and gives that clause the decisive weight. This is the kind of reasoning the experiment set out to measure.

Paper's attention picture (Figure 7): much of the attention in the last layer goes to the comma and the separator token. This is a useful map of information flow, but as the paper warns (citing Jain and Wallace, 2019), it is not a full explanation of the decision.

Part 8. Self-test

Try each question before looking at the answers on the next page. Use the real-world picture first, then the formal idea.

#	Question
1	Why can word counts alone fail on “not good”?
2	What does TF-IDF give a high weight to, and why does “movie” get weight 0 in the toy example?
3	What is the advantage of word vectors over one-hot word counts, and what is their main limitation?
4	Why was the recurrent network slower to train than the Transformer?
5	In your own words, what do query, key and value do in attention?
6	Why is a position signal needed, and what would go wrong without it?
7	Why does multi-head attention use several heads instead of one large head?
8	What is the role of the residual connection?
9	What is the difference between pre-training and fine-tuning?
10	What does a cross-entropy loss of 0.105 tell us compared with 1.61?
11	What is the purpose of warm-up in fine-tuning?
12	Why is distillation useful for building smaller models?
13	What do precision and recall each measure?
14	Why does a bootstrap interval matter when comparing two scores of 88.8% and 85.8%?
15	What do the two counts b and c measure in McNemar's test?
16	Why does the learning curve show the Transformer's advantage most clearly with few labels?
17	What is the trade-off between accuracy and latency in the paper?
18	How can a model classify sentences with no labelled examples?

Answers and short explanations

#	Answer
1	“Not” is a small word that flips the meaning of “good”. A word count sees “good” as positive and cannot tell that the position of “not” changes it.
2	TF-IDF gives high weight to words that are frequent in this sentence and rare across the collection. “Movie” appears in every sentence, so it tells us nothing about the difference between sentences, and its idf is ln(3/3) = 0.
3	Word vectors place similar words close together, so the model knows “coffee” is close to “tea”. Their limitation: each word has only one vector, even when the word has several meanings.
4	Each step of a recurrent network must wait for the previous one, so the work cannot be done in parallel. The Transformer processes all positions at once.
5	Query: what this word is looking for. Key: what each word offers to be matched. Value: the information passed on when it is matched. The query is compared with every key, and the weighted values are summed.
6	Attention alone treats the sentence as a bag of tokens. Without position, “man bites dog” and “dog bites man” would look identical to the model.
7	Each head can specialise in a different relation (for example, negation or adjective–noun links). Several small heads together cover more relations than one head of the same total size.
8	It keeps the original information flowing through the layers, so gradients can reach early layers in deep networks and the model can refine rather than replace information.
9	Pre-training learns general language from large unlabelled text. Fine-tuning adapts that model to one labelled task with a short extra training step.
10	A loss of 0.105 means the model gave the correct label a high probability (about 0.9). A loss of 1.61 means it gave the correct label about 0.2, which is a large error.
11	Warm-up starts with a tiny learning rate so that the pre-trained weights are not damaged by large updates at the beginning, then increases the rate to its peak.
12	A smaller student learns to imitate the output of a large teacher, which transfers much of the teacher's knowledge into a model that is cheaper to run.
13	Precision: of the predictions labelled positive, how many were right. Recall: of the true positives, how many the model found.
14	The two scores are close enough that the difference could be due to which sentences happened to be in the test set. The interval shows the likely range, so the comparison must be checked statistically.
15	b is the number of sentences only model A gets right, and c is the number only model B gets right. Their imbalance is the evidence for a difference.
16	With few labels, the pre-trained model already knows most words and their relations, while the TF-IDF model has seen very few examples of many words. The gap shrinks as labelled data grows.
17	The Transformer is more accurate but slower and bigger. For example, DistilBERT is about 650 times slower per sentence than the SVM but scores about 8 points higher.
18	A zero-shot NLI model turns the labels into hypotheses and checks which one the text supports. It needs no sentiment-labelled data, but it is the slowest model in the paper.

Final tip before presenting

For every concept, be ready to say three things: the real-world picture, what the model does with it, and what number in the paper shows it. That order makes an explanation easy to follow for an audience that is new to the topic.